

CODEFEST AD ASTRA 2026

Documento tecnico — Etapa 1: Base de Conocimiento Vectorial

1. Preprocesamiento y extraccion de texto

Cada formato de origen tiene un extractor dedicado que produce un objeto comun (*RawDocument*) con el texto crudo y metadata auxiliar: **PDF** (pdfplumber, preservando el orden de lectura por pagina y con deteccion heuristica de boilerplate — líneas idénticas repetidas en $\geq 30\%$ de las paginas de un documento de 3 o mas paginas se descartan como cabecera/pie de pagina), **HTML** (trafilatura, que separa el contenido principal del menu de navegacion, anuncios y pie de pagina, y preserva encabezados como senales Markdown), **JSON** (mapeo explicito de campos conocidos como *title/body_paragraphs/body_text*, con fallback generico para esquemas no anticipados), **CSV/XLSX** (cada fila se convierte en pares *columna: valor* y se trata como unidad atomica de fragmentacion) e **imagenes** (OCR opcional via pytesseract, con degradacion controlada si el binario de tesseract no esta disponible). La limpieza posterior (normalizacion Unicode NFC, remocion de caracteres de control, deteccion de idioma) es deliberadamente conservadora sobre el contenido — sin lowercasing ni stemming — porque la evaluacion compara el campo *text* de forma textual contra el ground truth.

2. Estrategia de chunking y justificacion

Se implemento una estrategia **hibrida estructural + oracional con solapamiento**, en tres niveles:

- **Estructural:** el texto se separa primero por encabezados Markdown (que HTML/JSON/MD ya traen marcados por el extractor); si el documento no tiene encabezados (caso tipico de PDF), se trata como una unica seccion.
- **Segmentacion de oraciones multilingue:** dentro de cada seccion, se usa *pysbd* (reglas linguisticas dedicadas) para espanol e ingles, y el parser entrenado de spaCy (*pt_core_news_sm*) para portugues, que *pysbd* no cubre nativamente.
- **Empaquetado voraz por presupuesto de tokens** (~280 tokens, margen bajo el limite de 512 del encoder), contando tokens con el tokenizer real del encoder de indexacion — no una aproximacion por palabras — con **solapamiento de 1 oracion** entre chunks consecutivos para no perder ideas que caen en la frontera.

La unidad minima que se mueve entre chunks es siempre una oracion completa: el empaquetador nunca corta un caracter a mitad de oracion, cumpliendo el requisito de completitud linguistica (sec. 3.3). Esta garantia se verifica automaticamente en *tests/test_chunking.py*. La unica excepcion estructural es CSV/XLSX, donde cada fila (no una oracion en lenguaje natural) se indexa como chunk atomico, tal como permite explicitamente la especificacion.

3. Encoder(s) seleccionado(s) y criterios de eleccion

Encoder primario: **sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2** (arquitectura encoder tipo BERT, licencia Apache 2.0).

Criterio	Justificacion
----------	---------------

Soporte multilingue	Cubre nativamente español, inglés y portugués (50+ idiomas), necesario porque el corpus y las consultas mezclan los tres.
Dimensionalidad	384 dimensiones: balance razonable entre costo de almacenamiento/busqueda y expresividad para el volumen esperado del reto.
Longitud máxima de entrada	512 tokens; el presupuesto de chunking (~280 tokens) deja margen suficiente.
Licencia	Apache 2.0, uso libre sin restricciones para el contexto del reto.
Eficiencia computacional	Modelo liviano (backbone MiniLM), adecuado para los recursos de cómputo limitados del equipo.
Riesgo de reproducibilidad	Ya validado en el material de apoyo entregado por el equipo organizador (01_embeddings.ipynb, 02_rag.ipynb), reduciendo el riesgo de incompatibilidades de último momento.

No se emplea ningún modelo decoder/generativo en ninguna etapa de indexación o recuperación (sec. 4.2 y 8.3): en particular, se descarto explícitamente HyDE (requiere un LLM para generar una respuesta hipotética) y el reranking con LLM. El uso de un *cross-encoder* (arquitectura encoder, no generativa, demostrado en el material de apoyo) quedó implementado detrás de un flag desactivado por defecto, como exploración documentada, para no depender de él en el flujo evaluado.

4. Índice FAISS

Se utiliza **IndexFlatIP** sobre vectores normalizados a norma unitaria, equivalente a similitud coseno (sec. 5.2 y 8.2). Es la opción recomendada por la propia especificación para el volumen de documentos esperado en esta etapa del reto: garantiza resultados exactos (sin la pérdida de exactitud de IVFFlat/HNSW) y su costo computacional es totalmente asumible a esta escala. El identificador interno de FAISS se mantiene alineado línea a línea con *metadata.jsonl* (mismo orden de inserción que de escritura), invariante verificado en *tests/test_faiss_alignment.py*. Si el corpus real de ADL resultara mucho mayor al esperado, *src/config.py* centraliza el punto donde migrar a un índice aproximado sin tocar el resto del pipeline.

5. Grafo de conocimiento (componente bonus)

NER: se usa el componente de reconocimiento de entidades ya entrenado en los modelos spaCy *es/en/pt_core_news_sm* (licencia MIT, arquitectura no generativa), filtrando tipos de entidad puramente numéricos/temporales (fechas, cantidades, porcentajes) para quedarse con personas, organizaciones, lugares y otras entidades nombradas relevantes.

Extracción de relaciones: heurística basada en dependencias sintácticas (permitida explícitamente por la especificación como alternativa a un modelo entrenado de RE): para cada oración con al menos dos entidades, se toma el verbo/auxiliar entre un par de entidades consecutivas como etiqueta de relación (o el verbo raíz de la oración si no hay uno intermedio), con una relación genérica de co-ocurrencia como último recurso.

Construcción e integración: las tripletas se acumulan en un *networkx.MultiDiGraph*, cada arista con *doc_id*, *chunk_id* y la relación como evidencia trazable, exportado a GraphML. En recuperación, las mismas entidades se detectan en la consulta, se buscan sus vecinos de primer orden en el grafo, y los chunks vinculados se fusionan con el ranking vectorial mediante Reciprocal Rank Fusion (el grafo se trata como un índice adicional, sec. 8.5), detrás de un flag *--use-graph* opcional en *generador.py*.

6. Modulo de recuperacion

- Mismo encoder para indexacion y consulta; vector de consulta normalizado igual que el indice.
- Sobre-recuperacion de candidatos antes de aplicar post-filtros de metadata (fenomeno, formato, idioma) o de umbral de similitud coseno (sec. 8.7).
- Agregacion a nivel documento por *max pooling* (configurable a suma o media) sobre un pool de candidatos mas amplio que los 10 fragmentos mostrados, para que la relevancia de un documento no dependa solo de si su mejor chunk aparece en el top-10 (sec. 8.6).
- Reciprocal Rank Fusion ($k_0=60$) como mecanismo unico de combinacion, reusado tanto para multiples encoders como para vector+grafo (sec. 8.4).
- Limite de 250 palabras por fragmento aplicado dividiendo en limites oracionales completos cuando un chunk lo excede; los sub-fragmentos comparten *chunk_id* y ocupan su propio *rank* (sec. 9.2.1).

Ninguna etapa de recuperacion usa un modelo generativo: solo vectores, puntuaciones de similitud coseno y aritmetica sobre metadata.

7. Limitaciones conocidas y decisiones documentadas

- **Corpus de ejemplo sintetico:** el entorno de desarrollo usado para construir este pipeline bloquea (proxy de salida, politica 403) la descarga de documentos reales de observatorios/centros de pensamiento. El corpus en *corpus_ejemplo/* (15 documentos, PDF/HTML, ES/EN/PT) fue redactado a mano, tematicamente inspirado en fuentes reales conocidas (SIPRI, ESA Space Debris Office, CEPAL, Instituto Kroc, entre otras) pero NO es una copia ni cita textual de ningun documento real. Se reemplaza integramente por el corpus oficial de ADL sin cambios en *src/*.
- **huggingface.co tambien bloqueado** en el mismo entorno: no fue posible descargar los pesos reales del encoder ni el modelo de NER de HuggingFace originalmente considerado para el grafo. El encoder esta detras de una interfaz intercambiable (*SentenceTransformerEncoder* real vs. *HashingFakeEncoder* determinista de pruebas); toda la suite de tests y las corridas de demostracion de este documento usan el encoder de pruebas, que valida la mecanica de indexacion/recuperacion/formato de salida pero NO la calidad semantica de la recuperacion. El indice con embeddings reales debe generarse corriendo *scripts/build_corpus_index.py* en un entorno con acceso normal a internet antes de la entrega final.
- **PBF (mapas):** no implementado en esta iteracion; es un formato nicho que requiere una libreria dedicada (osmium/pyrosm) y no se considero razonable fabricar datos geoespaciales sinteticos. La interfaz del extractor esta lista para completarse cuando el equipo tenga archivos PBF reales con los que validar el parseo.
- **Campo fuente:** se deriva hoy del nombre de archivo o la URL detectada por el extractor HTML. Es la clave real de emparejamiento con el ground truth a nivel documento (sec. 10.2.1), y esta aislada en una unica funcion (*derive_fuente()*) para ajustarla facilmente si ADL usa otra convencion.
- **Fusion de chunks cortos adyacentes** (sec. 9.2.1, mencionada como posible, no obligatoria) no esta implementada: solo se implemento la division obligatoria de chunks que exceden 250 palabras.
- **Boilerplate de PDF:** la heuristica de lineas repetidas ($\geq 30\%$ de paginas) requiere documentos de 3 o mas paginas; PDFs mas cortos no reciben este filtro.